

Five Silent Measurement-Validity Failures in a Generative-Model Evaluation Project

Gaurav Gandhi
Independent Researcher

Abstract

A promotion decision collapsed from +0.040 (3.18 SEM) to +0.0078 (0.58 SEM) once independent per-axis judge scoring replaced a correlated multi-axis call, and a “LoRA loses to base” verdict reversed to “viable at adapter weight 0.3-0.5” once the same comparison was run at more than one operating point instead of only the library default. Both reversals came from the same single evaluation project – scoring two SDXL LoRA style adapters and five base-model image-generation families with a local vision-language judge – which produced five distinct silent measurement-validity failures over its course, each capable of standing as an unchallenged finding if left unaudited. We separate the five into a value-validity class (malformed or out-of-range values, catchable by automated range/uniqueness checks) and a semantic-validity class (well-formed values that silently answer the wrong question, not catchable by value checks alone), and show that each automated safeguard built in response to one defect failed to generalize to the next. Three further confirmed instances, found later in an unrelated subsystem (shell orchestration for a cloud GPU run), corroborate the same class recurring outside the original harness – reported separately from the five, not folded into the count. A sixth, structurally similar anomaly is reported and left explicitly as an open, unresolved candidate, not a confirmed defect. We argue the taxonomy, and the discipline of re-auditing already-accepted conclusions rather than trusting an accumulating stack of automated checks, generalizes beyond one VLM-judge harness.

1. Introduction

This paper’s claim is narrow and stated with its scope attached: on one evaluation project, five distinct ways a measurement can be silently wrong were each found in turn, and the automated defenses built after each one did not generalize to catching the next. That is the finding worth reporting — not the specific numbers behind any one adapter’s publication decision, which is already closed and documented elsewhere (`docs/MODEL_VERDICT.md`).

The project’s evaluation harness scores generated images against a local VLM (Ollama, `qwen2.5v1:7b`) on a fixed rubric — `style_adherence`, `figure_preservation`, `artifact_absence` — using pre-registered decision rules (a diff/SEM threshold checked *before* results are read) across two style-adapter LoRAs (ukiyo-e, Pattachitra) and five non-adapter base-model families. Over the course of running that harness, five silent defects were found, in this order: a context-window truncation that silently dropped judge calls (§4.1); a stale GPU-memory counter that reported an identical, physically-impossible reading across independent runs (§4.2); a CUDA context corruption that could have retroactively poisoned images generated before the crash that revealed it (§4.3); a judge prompt hardcoded to name one style domain regardless of which domain was actually being evaluated (§4.4); and a script that silently reused a stale, wrong-question score for one arm of what was presented as a fresh, apples-to-apples comparison (§4.5). A sixth, structurally similar anomaly

is reported separately in §7 as a candidate, not a finding, because it remains open which of three plausible mechanisms explains it.

Why this is reported as a standalone result rather than a footnote to any one model’s verdict. Two of the five defects (four and five) were found only because a *previous conclusion the project had already written down and accepted* was re-opened and checked against its own assumptions — not because anything about the number looked wrong on its face. A project’s confidence in a result is not evidence that the result is uncorrupted; the discipline that caught these defects was routinely re-auditing already-accepted conclusions, not trusting a clean-looking pipeline more as it accumulates more checks.

1.1 Roadmap

Section 2 describes the evaluation harness and the pre-registration discipline this project used throughout. Section 3 gives the taxonomy distinguishing value-validity from semantic-validity defects. Section 4 reports each of the five defects: mechanism, discovery path, and the consequence had it gone unaudited. Section 4.6 reports three further, confirmed bugs found later in a different subsystem (GCP shell orchestration) — corroborating evidence that the taxonomy generalizes beyond VLM judging, explicitly not a sixth/seventh/eighth entry in the five-defect count. Section 5 reports the two concrete verdict reversals this project’s defects produced, with the numbers on both sides of each reversal. Section 6 discusses the pattern across all five — specifically, why each automated check built in response to one defect failed to generalize to the next. Section 7 reports the sixth, unresolved candidate anomaly. Section 8 states limitations plainly. Section 9 gives the reproducibility/provenance trace.

2. Background — The Evaluation Harness

`scripts/model_verdict_harness.py` scores every generated image on three axes via one Ollama call per axis (`SINGLE_AXIS_JUDGE_PROMPTS`) rather than one multi-axis call per image — a design adopted specifically *because* an earlier multi-axis, single-call design was found to be vulnerable to judge halo effects (all three axes anchoring to one overall impression; `docs/MODEL_VERDICT.md` §4.5). Every comparison in this project pre-registers its primary endpoint, promotion threshold, and guardrail axes *before* the run executes (`docs/AB_PREREGISTRATION.md`, `docs/PATTACHITRA_AB_PREREGISTRATION.md`, `docs/WEIGHT_SWEEP_PREREGISTRATION.md`), and states its decision rule as fixed in advance — a null or negative primary result is reported as such, not treated as license to keep searching for a threshold that passes.

Two LoRA style adapters (ukiyo-e, trained on SD 2.1/SDXL; Pattachitra, SDXL only) and five non-adapter base families were scored on this harness across the life of the project. The defects reported here surfaced across that full span, not in one isolated script.

3. Taxonomy: Value-Validity vs. Semantic-Validity Defects

The five defects split cleanly into two classes, and the split matters because it predicts which automated checks can catch which defects.

Value-validity defects — the measured value itself is malformed, out of range, missing, or statistically impossible, independent of what question was asked. These are catchable by automated checks that inspect the value alone: range validation, null/error-rate checks, degenerate-output detection (NaN, pure-black/white, near-uniform images), physical-plausibility checks (a VRAM reading exceeding the card’s installed memory), and per-record uniqueness assertions. Three of the five defects (context-window truncation, phantom VRAM counter, CUDA context corruption) are in this class, and this project did in fact build automated checks for all three after finding them (`model_verdict_harness.py`’s CUDA pre-flight probe, per-record uniqueness assertion, degenerate-image detector, and judge-score range validator).

Semantic-validity defects — every individual value returned is well-formed, in range, and internally consistent, but the value answers a different question than the one the downstream conclusion assumes it answers. These are *not* catchable by inspecting the value alone; they require inspecting the measurement instrument’s own logic (what question was actually sent, what data was actually loaded) against the claim built on top of it. Two of the five defects (hardcoded judge question, stale reused reference arm) are in this class, and no automated check built in response to the first three defects caught either of them — both were found by direct code inspection, prompted by different triggers (see §6).

Defect class	Example (this project)	Catchable by automated value checks?	What kind of check would catch it
Value-validity	Context-window truncation, phantom VRAM counter, CUDA corruption	Yes	Range/null checks, physical-plausibility bounds, degenerate-output detection, uniqueness assertions
Semantic-validity	Hardcoded judge question, stale reused reference arm	No	Direct source inspection of the instrument; domain-parameterization tests (assert the actual request text names the caller’s stated domain); cross-report reconciliation (assert the same logical quantity — same model, axis, n — agrees everywhere it’s reported, or traces to one shared source)

This taxonomy is not scoped to VLM judging specifically, though every defect above happens to come from that harness. §4.6 reports three further confirmed bugs from a later, unrelated subsystem (GCP shell orchestration, no VLM/judge/HF involvement at all) that fit the semantic-validity description precisely — a well-formed-looking outcome (a captured exit code,

an apparently-completed script) that silently answers a different question than the one relied on. Those three are corroborating evidence for the taxonomy’s generality, not a sixth/seventh/eighth entry in the table above — the table’s row count and the paper’s “five” stay as reported in §4.

4. Findings — Five Defect Classes

Each subsection follows the same structure: what the defect was, how it was actually found, and what wrong conclusion it would have produced if it had gone unaudited. Full detail and code references are in `docs/MODEL_VERDICT.md` §7.7 (source table this section expands). §§4.1–4.5 are the five; §4.6, despite the section number, is corroborating evidence from a separate subsystem, not a sixth member of this section’s own title count.

4.1 Defect 1 — Judge context-window truncation (value-validity)

What it was. Ollama’s default served context window (4,096 tokens) was too small for a judge prompt plus a high-resolution image’s token count on some inputs, causing the request to fail with a 400 error (`exceed_context_size_error`) rather than return a score.

How it was found. A real, literal 400 error surfaced during Pattachitra training-corpus curation — this was not predicted or hunted for in advance; the harness simply started failing on inputs that happened to cross the token threshold.

Unaudited consequence. Every judge call for a sufficiently large prompt+image combination would fail outright — not a wrong score, a *missing* one. This silently reduces `n` for whichever records happen to exceed the context window, understating the true sample size without any corresponding downward revision of a confidence claim built on the (falsely larger) assumed `n`.

Fix. `num_ctx=8192` set explicitly in `_ollama_generate_json`, verified against the actual Ollama error body rather than assumed to be an unrelated GPU-contention issue.

4.2 Defect 2 — Phantom VRAM counter (value-validity)

What it was. `torch.cuda.max_memory_allocated()` reported an identical 11.186 GB peak across three separate runs on an 8.589 GB local card — a physically impossible reading, since no run on that hardware can allocate more memory than the card has installed.

How it was found. Manual forensic investigation (`docs/LATENCY_ROOT_CAUSE.md`), triggered by an unexplained $5.6\times$ latency variance between runs — not by any automated check. The root cause was a stale, never-reset counter: `reset_peak_memory_stats()` was missing, so each “new” reading carried over a peak from an earlier, larger run (plausibly a contended GCP L4 instance, not the local card being measured at all).

Unaudited consequence. Taken at face value, “every run used 11.186 GB” would have been reported as a hardware-fit finding for a card that cannot physically hold that much — a false capacity/fit conclusion built on a counter that was never actually being measured fresh each run.

Fix. `reset_peak_memory_stats()` added to `scripts/eval.py` before each timed run.

4.3 Defect 3 — CUDA context corruption with retroactive-poisoning risk (value-validity)

What it was. A `curated500` generation attempt crashed with `RuntimeError: CUDA error: an illegal memory access was encountered` after 22 of 90 images had been generated; the poisoned

CUDA context then failed all 68 remaining attempts in the same process.

How it was found. Not by the crash alone — that part was obvious. It was found by a *skeptical re-audit of a prior turn’s own already-accepted conclusion*: an earlier turn had already accepted the Pattachitra `figure_preservation` finding built partly on this run’s data, and a later turn specifically asked whether the crash could have silently degraded the 22 images generated *before* it, not just the 68 that failed outright afterward.

Unaudited consequence. Left unaudited, the working assumption would have been “the crash only cost 68 failed attempts, nothing else” — an assumption that was never actually verified, resting on architectural reasoning (“a crash can’t retroactively corrupt already-saved data”) rather than the direct pixel-level and visual confirmation this project eventually ran. The audit checked file-mtime provenance to identify the 22 at-risk images, ran a pixel-level integrity scan (mean/std/NaN/pure-black/pure-white per image), and did a direct visual inspection of the boundary images including the last one generated before the crash. All 22 were confirmed clean; no regeneration was warranted — but the assumption had to be checked, not inferred from architecture.

Fix. A permanent CUDA pre-flight health probe, a per-record uniqueness assertion, and degenerate-image detection (NaN/black/white/near-uniform) added to `scripts/model_verdict_harness.py`, aborting a run rather than writing a poisoned record.

4.4 Defect 4 — Hardcoded judge question (semantic-validity)

What it was. The `style_adherence` judge prompt in `model_verdict_harness.py` was hardcoded to ask whether the image looked like ukiyo-e, regardless of which domain the caller was actually scoring. Pattachitra evaluation scripts reused this prompt unmodified, so all 360 Pattachitra `style_adherence` records — the primary endpoint for that adapter’s publication decision — were generated by asking the judge about the wrong style entirely.

How it was found. Direct code inspection, while *designing a new, unrelated positive-control script* to validate that the judge could discriminate style at all — found before any Pattachitra-domain result from that new control was even read. None of the three integrity checks built in response to defects 1–3 (the CUDA health probe, degenerate-image detection, and per-record uniqueness assertion) caught this, because all three of those checks validate that a returned *value* is well-formed; none of them inspect what *question* produced that value.

Unaudited consequence. A confidently-reported “no style-adherence lift, and no significant effect either way” finding — a plausible-looking null that was in fact measuring an entirely different question, indistinguishable from a genuine null without the code-level check that finally caught it.

Fix. `score_vlm_judge` now requires an explicit `style_question` keyword-only argument with no default, so the harness itself carries zero style-domain knowledge; every caller passes its own domain’s question explicitly. Nine new regression tests (`TestStyleQuestionIsNeverHardcoded`) assert: no default exists (a `TypeError` if the argument is omitted); the actual Ollama request text contains the caller’s stated domain; two different domains produce two different requests; and the other two axes (`figure_preservation`, `artifact_absence`) remain domain-neutral.

4.5 Defect 5 — Stale reference arm reused across two comparisons (semantic-validity)

What it was. The judge positive-control script (`scripts/judge_style_positive_control.py`) loaded the `sdxl_base` reference arm’s Pattachitra `style_adherence` scores once, from an old file scored under defect 4’s wrong (ukiyo-e) question, and reused that same value unchanged for a

second, supposedly-independent row of the same comparison — the “corrected prompt” row — which should have re-scored the base arm fresh under the corrected question instead.

How it was found. A genuine cross-document contradiction: the same logical quantity — `sdxl_base`’s `Pattachitra style_adherence` mean, `n=90` — was reported as two different numbers, 0.3533 and 0.8883, in two different places that should have agreed exactly. That contradiction, not an a priori suspicion of this specific script, prompted a direct read of `judge_style_positive_control.py`’s source, which located the stale-reuse bug.

Why the defect-4 check did not catch this. The domain-parameterization tests built for defect 4 assert that the *question text* sent to the judge names the correct domain. In this script, the question text was never wrong — the bug is that *stale data from a different, valid measurement* was silently substituted into what was presented as a second, independent measurement. A check that validates question text has nothing to say about which array of already-computed scores a script chooses to load.

Unaudited consequence. A confident, decisive-looking PASS (`diff/SEM = +12.005`) that never actually compared two arms scored under the same question — the “corrected prompt” row silently mixed a freshly-scored real-art arm against a stale, wrong-question base arm. The corrected, fair comparison reverses the verdict entirely (FAIL, `-3.781×SEM`), which changed the *Pattachitra* publication reassessment downstream (§5.2).

Fix. `judge_style_positive_control.py` now scores the base arm fresh, per comparison, with no cross-run reuse.

4.6 Corroborating evidence from a different subsystem — not a sixth defect, not counted among the five

This subsection reports three additional, confirmed bugs found in a later, separate piece of work on this project (orchestrating a GCP evaluation run for a sixth model family, **FLUX.1-schnell**) — and explicitly does not add them to the five above. The paper’s title and count (§1, §10) are unchanged; these three are reported here because they recurred in a *different subsystem* — bash shell orchestration, with no VLM judge, no Ollama, and no HF model card involved at all — during the writeup of the five defects above, which is evidence the underlying pattern is not an artifact specific to one VLM-judge harness.

The bugs. A GCP startup script (`scripts/gcp_startup_flux_eval.sh`) probes one loading config, and falls back to a second (NF4-quantized) config if the first fails or is too slow — a correctly-designed safety net, written and reviewed before any real run. All three bugs are the identical mechanism recurring at three points in the same script: the script runs under `set -e -o pipefail`, and at each point a failure that the fallback logic was specifically written to handle instead silently defeated that logic by terminating the whole script one line before the fallback check could run.

1. The probe’s Python process OOM’d on a real GCP L4 run (a genuine, reproducible CUDA out-of-memory error — the 23.78GB transformer module doesn’t fit as a single resident unit in ~22GB usable VRAM even with the rest of the pipeline offloaded, confirmed twice). That OOM propagated through a piped `tee` command with no `|| true`, and `pipefail` turned the pipeline’s exit status non-zero, aborting the entire script under `set -e` before the already-written “if the probe produced no result, use the fallback” check ever ran.
2. Fixing (1) was not sufficient: the very next line, `grep -oP '...' | tail -1`, *also* exits non-zero under `pipefail` when `grep` finds no match — which is exactly what happens when the probe crashed and never printed a result, the one case this whole fallback exists to handle.

This was found only by re-running the fixed script on GCP and watching it die at the identical point again, one line further in.

3. A third instance was caught by direct review of an in-progress fix, before it was ever deployed: capturing the main harness process’s exit code as `wait "$EVAL_PID"; EVAL_EXIT=$?` has the identical `set -e` exposure as (1) and (2) — a non-zero `wait` would abort the script before `EVAL_EXIT=$?` could run. The naive fix (`wait "$EVAL_PID" || true`) was drafted, then caught as wrong on re-reading it: `|| true` makes the *whole compound command* succeed, so the next line’s `$?` would read `true`’s own exit code (0) instead of the harness’s real one — a well-formed, in-range, entirely plausible-looking “success” signal that would have silently reported every harness failure as a success, exactly the semantic-validity pattern defects 4 and 5 describe (§3), just one line away from shipping. Fixed instead with `EVAL_EXIT=0; wait "$EVAL_PID" || EVAL_EXIT=$?`, which preserves the real code.

Precise count, not inflated. These are three confirmed, verified bugs (the mechanism for each is definitively understood, with no ambiguity — unlike §7’s candidate sixth anomaly, which remains genuinely unresolved among three competing hypotheses). But they are not uniform in how they were caught, and that distinction matters and is stated rather than smoothed over: (1) and (2) were each confirmed by a *live failure* on the actual GCP run — the script visibly died both times, and the second failure was only found because the first fix was tested for real, not assumed sufficient. (3) never manifested in any run; it was caught by re-reading a draft fix before deploying it, the same discovery mode as defect 4 (§4.4) — direct code inspection prior to reading any result. Two live-failure instances plus one pre-deployment catch is reported as exactly that, not rounded up to “three equivalent occurrences” and not merged into the candidate sixth class, which is a different kind of open question (an unresolved measurement anomaly) rather than a verified control-flow bug.

Why this strengthens the paper rather than padding it. All three are the identical mechanism (`set -e/pipefail` silently defeating fallback logic written for exactly the failure that triggers it) recurring in a subsystem with zero overlap with the VLM-judge harness that produced defects 1–5 — different language considerations (bash control flow, not a judge prompt or a Python score), different failure surface (a safety net for a GPU OOM, not a style question or a reused reference score), same underlying shape: an automated safeguard that looks like it will catch a known failure mode, and silently doesn’t. That recurrence in an unrelated subsystem is evidence the pattern is a property of how automated safeguards get defeated, not a quirk of one project’s one evaluation harness — addressed further in §6 and §8.

5. Reversals as Evidence

The five defects above are not abstract methodology observations — two of them changed an already-written verdict on an already-generated model. Both reversals are reported here with the numbers on both sides, because the size of the swing is itself the evidence that these were not minor corrections.

5.1 Ukiyo-e SDXL LoRA: promotion withdrawn (defects 1 and the halo-effect risk that motivated independent-axis scoring)

The curated-retrain vs. published-checkpoint comparison was first scored under a single Ollama call per image covering all three axes at once — a design later shown to be vulnerable to judge

halo effects, where one overall impression can bleed into all three axis scores. Under that regime, the primary endpoint (**artifact_absence**, paired diff, **n=90**) cleared the pre-registered $2\times\text{SEM}$ promotion bar decisively:

Regime	Paired diff	Paired SEM	diff/SEM
Correlated, single-call (docs/MODEL_VERDICT.md §4.3)	+0.0400	0.0126	+3.182
Independent-axis, full n=90 rescore (docs/MODEL_VERDICT.md §4.6)	+0.0078	0.0133	+0.583

Rescoring the identical 180 images (90 published, 90 curated) with three independent Ollama calls per image — one per axis, no shared impression across axes — collapsed the effect from $3.18\times\text{SEM}$ to $0.58\times\text{SEM}$, below the pre-registered promotion threshold. The 95% CI on the true diff under the trusted regime is $[-0.0184, +0.0339]$; its upper bound sits below the originally claimed +0.0400, meaning the design has enough precision to rule out the original claim, even though it lacks the power ($\text{MDE} \approx 0.0374$ at 80% power) to fully characterize a smaller true effect. A follow-up power audit found that resolving the remaining ambiguity to the precision of the observed effect (~ 0.008) would require **$\approx 1,963$ paired samples — roughly $22\times$ this study’s $n=90$** — impractical at zero-cost local scale. The published checkpoint-1000 was not re-promoted on this data; the promotion decision was withdrawn, not silently downgraded.

5.2 Pattachitra SDXL LoRA: “loses to base” reversed to “viable at a documented low weight”

The Pattachitra adapter was first evaluated only at **adapter_weight=1.0** (the library default), where every checkpoint significantly regressed **figure_preservation** against **sdxl_base** ($-5.5\times\text{SEM}$ to $-7.8\times\text{SEM}$, visually confirmed on specific prompts where the LoRA drops the requested human figure entirely) and neither checkpoint showed a style lift. The verdict at that single operating point was unambiguous: do not publish.

A subsequent adapter-weight sweep (both checkpoints, weight values below 1.0) found four operating points — weight 0.3 and 0.5, both checkpoints — clearing a joint pre-registered criterion (**style_adherence** diff/SEM $> +2.0$ AND **figure_preservation** diff/SEM ≥ -2.0 at the same weight):

Checkpoint / weight	style_adherence diff/SEM	figure_preservation diff/SEM
checkpoint-500, weight 0.3	up to +3.622	not regressed
checkpoint-1000, weight 0.5	up to +3.482	not regressed
checkpoint-500, weight 0.5	—	up to +2.080 (modest improvement)

At **weight=1.0** the same two checkpoints still regress both axes decisively — that fact is retained on the card, not superseded by the lower-weight finding. The reversal here is not “the first evaluation was wrong”; it is that evaluating at exactly one operating point (the library default) produced a true-but-incomplete verdict, and the sweep was required to surface the region where the adapter is actually usable. This reversal survived its own defect: an interim FAIL on the style-adherence

positive control (§4.5’s category of bug, applied to a different control script) briefly cast doubt on the sweep’s `style_adherence` half before a corrected, symmetric control design confirmed the judge could in fact discriminate Pattachitra style ($+17.951 \times \text{SEM}$ and $+28.659 \times \text{SEM}$ against two independent off-style contrasts).

6. Discussion — Why Each Check Did Not Generalize

The pattern, not just the count, is the finding. Defects 2 and 3 were each caught by manual, one-off forensic investigation triggered by something looking anomalous — an impossible number, an unexplained latency variance — and depended on a person noticing that something felt wrong enough to dig in. Defect 4 was caught by neither an anomaly nor an automated check; it was caught by directly reading the source of the measurement instrument itself, while building an unrelated new tool. Defect 5 was caught by yet a different mode again: a cross-document contradiction (two different numbers reported for what should have been one identical quantity) that prompted a direct read of the script producing one of them.

Four discovery modes for five defects — anomaly-triggered forensic investigation (defects 2, 3), direct code inspection while building something unrelated (defect 4), and cross-document contradiction (defect 5) — and every automated check this project actually built in response to defects 1–3 (the CUDA health probe, degenerate-image detection, per-record uniqueness assertion, judge-score range validation) checks *value* validity. The check built specifically in response to defect 4 checks *question* validity — does the actual request text match the domain under test. Neither category of check, however thorough, can catch a *reused, stale result from a different, valid measurement* being silently substituted into a new comparison, which is exactly what defect 5 was. This is a distinct defect class again, not a variant of defect 4, and the project’s own conclusion is that it likely needs a further class of check — for example, an automated audit that flags whenever the same logical quantity (same model, axis, and `n`) is computed more than once across a project’s report files and asserts the values agree, or trace to one shared source, before either is used in a downstream conclusion. No such check has been implemented; it is logged here as a candidate for future harness work, not a completed mitigation.

A fifth discovery mode, found in the corroborating GCP-script bugs (§4.6): re-reading a draft fix before it ships, rather than after a live failure. Two of §4.6’s three bugs were caught the same way defects 2 and 3 were — a real failure, watched directly, on an actual run. The third was caught differently again: not by an anomaly, not by a contradiction, but by re-reading an about-to-ship one-line fix and recognizing it would silently report a failure as a success. This is close to defect 4’s mode (direct code inspection before any result is read) but distinct in timing — defect 4 was caught while building an unrelated tool; this one was caught by double-checking the very fix meant to close a bug just found, which is a narrower and more replicable habit (review your own fix before running it, not just after it fails) than “notice something looks off while working on something else.”

7. A Sixth, Unresolved Anomaly (Candidate, Not a Finding)

While confirming defect 5’s fix, a same-signed anomaly recurred independently in a second domain: both style domains’ straightforward positive controls showed real reference art scoring *below*

domain-prompted synthetic `sdxl_base` output — ukiyo-e: 0.9239 vs. 0.9389; Pattachitra: 0.7960 vs. 0.8883. Re-testing with a symmetric off-style-contrast design (real art vs. two genuinely different comparisons, not domain-prompted synthetic) passed decisively for both domains once tried, ruling out general judge blindness to either style. What remains open is narrower: why real photographed art scores below `sdxl_base`’s own domain-prompted synthetic output specifically. Two same-signed data points, one per domain, cannot yet distinguish between three competing explanations that would predict the same direction: (a) the judge rewards visually “clean” digital rendering over photographed physical artwork, independent of style; (b) the base model’s generations and the judge’s own question both derive from the same textual style description, so the generation may match the judge’s literal criterion more closely than an authentic photograph ever could, regardless of medium; (c) real reference photographs carry ordinary documentary-photography degradation (lighting, glare, cropping, print wear) that could depress a VLM’s judgment independent of any style-specific signal. This is logged as a methodology finding — real-vs-domain-prompted-synthetic comparisons are not safe to assume fair by default, even though the instrument itself is confirmed sound for both domains — and explicitly not claimed as a sixth confirmed defect. Distinguishing (a)/(b)/(c) is out of this project’s scope and would require a dedicated follow-up control (e.g., scoring digitally-scanned, never-photographed art against photographed art of the same style, isolating medium from prompt-vocabulary overlap).

8. Limitations

- **n=1 project.** All five defects and the sixth candidate come from a single evaluation project, with one VLM judge (`qwen2.5v1:7b`, local, zero-cost) and two style-adaptor domains. Whether this defect *rate* (five in one project) or this defect *taxonomy* generalizes to other generative-model evaluation pipelines is not tested here and should not be assumed. §4.6’s three corroborating bugs are one data point in favor of the *taxonomy* generalizing beyond VLM judging specifically (they recur in bash orchestration code with no judge involved) — this is still n=1 at the project level, and does not extend to a claim about defect rate, or about any pipeline this project has not touched.
- **VLM-judge dependence.** Every defect and every fix is scoped to a VLM-as-judge scoring design. A pipeline using a different scoring method entirely (human raters, a different automated metric family) would not necessarily exhibit the same failure modes, though the value/semantic-validity taxonomy (§3) is not obviously judge-specific.
- **The sixth anomaly is explicitly not a finding.** §7’s pattern has two same-signed data points, not an isolated mechanism — it is reported as an open methodology question, not a confirmed defect class, and should not be cited as a sixth entry in the defect table (§3, §4).
- **§4.6’s three GCP-script bugs are not a sixth/seventh/eighth defect either, for a different reason than the candidate anomaly above.** They are fully confirmed (the mechanism for each is definitively understood, unlike §7’s genuinely open question) but deliberately kept out of the paper’s count because they come from a different subsystem in a later, separate piece of work — reported as corroborating evidence for the taxonomy’s generality (§3, §4.6), not folded into “five.” Within those three, two were confirmed by a live, reproduced failure and one was caught by re-reading a draft fix before it ran — that distinction is stated in §4.6 rather than presenting all three as equivalent-strength evidence.
- **Discovery survivorship.** All five defects were, by construction, found — this project has no way to bound how many additional silent defects of a similar or different kind were not

found. The taxonomy in §3 describes what was caught and by what mechanism; it is not a claim that these two classes exhaust the space of possible silent measurement failures.

9. Reproducibility and Provenance

Every defect and figure in this draft traces to a specific section of `docs/MODEL_VERDICT.md` (the source of record) or a named script/report file in this repository:

Claim	Source
Defect table (six rows, five confirmed + one candidate)	<code>docs/MODEL_VERDICT.md</code> §7.7
Ukiyo-e correlated-regime headline (+0.0400, $3.182 \times \text{SEM}$)	<code>docs/MODEL_VERDICT.md</code> §4.3, <code>reports/lora_ab_30prompt.json</code>
Ukiyo-e independent-axis rescore (+0.0078, $0.583 \times \text{SEM}$)	<code>docs/MODEL_VERDICT.md</code> §4.6, <code>reports/lora_ab_30prompt_independent.json</code>
Ukiyo-e power/MDE audit (MDE 0.0374, $n \approx 1,963$ required)	<code>docs/MODEL_VERDICT.md</code> §4.7, <code>reports/lora_ab_power_audit.json</code>
Pattachitra weight=1.0 verdict (figure_preservation -5.5 to $-7.8 \times \text{SEM}$)	<code>docs/MODEL_VERDICT.md</code> §7.2–§7.3
Pattachitra weight-sweep operating points (0.3/0.5, both checkpoints)	<code>docs/MODEL_VERDICT.md</code> §7.4, <code>docs/WEIGHT_SWEEP_PREREGISTRATION.md</code>
Symmetric control reversal (+17.951 $\times \text{SEM}$, +28.659 $\times \text{SEM}$)	<code>docs/MODEL_VERDICT.md</code> §7.2 Addendum, §7.4
Sixth-anomaly figures (0.9239/0.9389, 0.7960/0.8883)	<code>docs/MODEL_VERDICT.md</code> §7.7 row 6
§4.6’s three GCP-script bugs (probe-pipeline, grep-extraction, EVAL_EXIT-capture)	<code>scripts/gcp_startup_flux_eval.sh</code> git history, commits <code>f012d16</code> , <code>7174875</code> , <code>fe4899d</code> ; <code>docs/FLUX_EVALUATION.md</code> §4

This draft is not itself pre-registered — it is a retrospective methodology writeup of decisions and results that were each pre-registered individually, at the time, in the documents listed above. No new measurement is reported here; every number is copied from, and traceable back to, an already-committed report file.

10. Conclusion

Five silent measurement-validity defects occurred over the course of one generative-model evaluation project, split evenly in kind: three that a value-level automated check can catch (and now does, in this project’s harness), and two that no such check can catch because the value returned was well-formed and the defect lived in what question was asked or what data was silently reused. The project’s own defenses improved incrementally and honestly — each new check targeted the specific failure mode that had just been found — but each new defect arrived from a different discovery mode than the checks already in place were built to catch. Two of the five defects reversed an

already-accepted conclusion outright: a promotion verdict that collapsed by more than $2.5 \times \text{SEM}$ under corrected scoring, and a “loses to base” verdict on a full-strength adapter that reversed to “viable and publishable” once evaluated at more than one weight. Neither reversal was found by a check running automatically at measurement time; both were found by treating an already-accepted result as something still worth auditing.